

AI Specialization 6: Robot Decision Making, Planning, and Coordination

AI Specialization 6: UIR

Decision making, planning, and coordination of autonomous systems of one or more robots.

Sources used: official CTU CourseWare page for B(E)4M36UIR, which lists lectures/labs on robotic paradigms, grid and graph planning, exploration, incremental planning, multi-goal/data-collection planning, curvature-constrained data collection, PRM, RRT, and risk-aware planning; CTU state-exam topic list; standard robotics planning material.

UIR in Plain English

UIR is robot planning in the physical world. The key difference from abstract planning is geometry, sensing, uncertainty, motion constraints, and coordination between robots.

1. Name the robot problem.

Control, path, exploration, tasks, or sampling.

2. Name the space.

World, grid, graph, or configuration space.

3. Give the method.

A*, D*, frontier, allocation, PRM/RRT.

Robot architectures

How sensing, planning, and acting are organized.

- Hierarchical: sense-plan-act.
- Reactive: fast behavior from perception.
- Hybrid: deliberative planner plus reactive layer.

Path and motion planning

Move from start to goal without collision.

- Configuration space represents robot poses.
- Grid/graph planning uses A* and variants.
- D*/D* Lite replan when the map changes.

Exploration

Plan to gain information, not just reach a known goal.

- Frontier = boundary between known free and unknown space.
- Information gain and entropy score useful observations.
- Exploration balances travel cost and new knowledge.

Multi-robot and multi-goal

Assign tasks and order visits.

- Centralized methods optimize globally but scale poorly.
- Decentralized methods scale but may be less optimal.
- Robotic TSP adds geometry and motion constraints.

Sampling planners

Avoid explicit full-space search in continuous spaces.

- PRM builds a roadmap.
- RRT grows a tree toward random samples.
- RRT*/PRM* add asymptotic optimality.

How to answer

Always mention physical constraints.

- What is the state/configuration?
- What is known vs. unknown?
- What guarantee does the planner have?

Exam sentence: Motion planning usually works in configuration space, where each point represents a full robot pose and obstacles become forbidden regions.

Three traps

- Path planning ignores some motion constraints; motion planning includes geometry/kinematics.
- Probabilistic completeness does not mean guaranteed success in finite time.
- RRT is not asymptotically optimal; RRT* is.

Use this page to separate robotics planning topics by what is being planned.

1. Robotic Paradigms and Control Architectures

Robot, embodiment, sensors, actuators

A robot is an embodied agent acting in the physical world.

Core parts:

- **Sensors:** measure robot/world state.
- **Actuators/effectors:** cause motion or physical interaction.
- **Controller:** maps perception/internal state to actions.

Embodiment means the robot has a physical body subject to geometry, dynamics, energy, time, uncertainty, and interaction constraints.

Sensor types:

- **Proprioceptive:** internal state, e.g. encoders, IMU, battery, joint positions.
- **Exteroceptive:** external world, e.g. lidar, camera, sonar, bumper, GPS.

Actuators include DC motors, servos, pneumatic/hydraulic actuators, propellers, wheels, legs, grippers.

Hierarchical paradigm

Pipeline:

$$Sense \rightarrow Plan \rightarrow Act.$$

Features:

- explicit world model,
- deliberative planning,
- long-horizon reasoning.

Advantages:

- systematic,
- can optimize globally,
- good for predictable environments.

Drawbacks:

- slow reaction,
- brittle if model is incomplete,
- frame problem and closed-world assumptions.

Reactive paradigm

Direct coupling from perception to action, often behavior-based:

$$Sense \rightarrow Act.$$

Examples: obstacle avoidance, wall following, potential fields, subsumption architecture.

Advantages:

- fast,
- robust to local changes,

- simple behaviors can be composed.

Drawbacks:

- weak global planning,
- can get stuck in local minima,
- harder to guarantee task completion.

Hybrid paradigm

Combines deliberative and reactive layers:

- high-level planner selects goals/subgoals,
- executive monitors execution,
- reactive layer handles fast control and safety.

Typical architecture:

Planner $\rightarrow$ *Sequencer/Executive* $\rightarrow$ *Behaviors/Controllers*.

Advantages:

- combines global reasoning with fast response,
- practical standard for autonomous robots.

Drawbacks:

- integration complexity,
- consistency between layers,
- debugging and timing issues.

Simple explanation

Hierarchical control thinks first and acts later; reactive control acts immediately; hybrid control uses both. Most real robots are hybrid because they need long-term goals and fast safety reactions at the same time.

2. Path and Motion Planning

Path vs. motion planning

Path planning: find collision-free geometric path.

Motion planning: find feasible trajectory respecting kinematics/dynamics, usually time-parametrized.

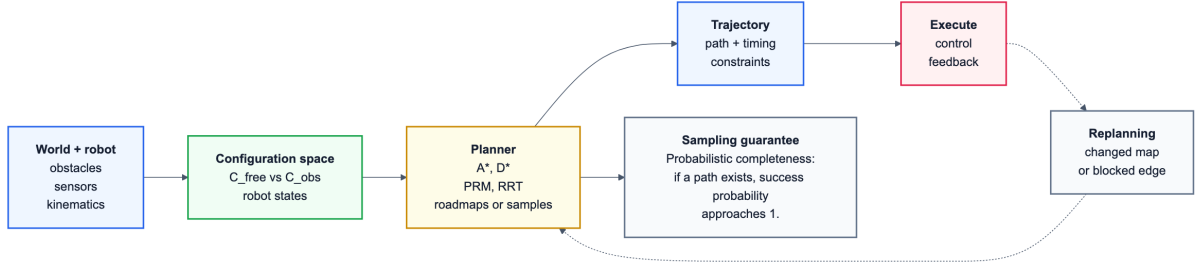


Figure 1: Robot planning pipeline: world and robot geometry are mapped to configuration space, planned over, executed, and replanned when the environment changes.

Configuration space

Configuration q fully describes robot pose/joint state. Configuration space:

$$\mathcal{C}$$

Obstacle region:

$$\mathcal{C}_{obs}$$

Free space:

$$\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{C}_{obs}.$$

Planning problem: find continuous path

$$\tau : [0, 1] \rightarrow \mathcal{C}_{free}$$

with $\tau(0) = q_{start}$ and $\tau(1) = q_{goal}$.

Simple explanation

Configuration space turns robot motion into point motion. Instead of reasoning about every point on a robot's body, we reason about one point in a higher-dimensional space where forbidden regions represent collisions.

Example

A circular mobile robot moving in a 2D map has configuration (x, y) if orientation does not matter. Obstacles in the workspace are inflated by the robot radius; then the robot can be treated as a point moving through the remaining free space.

Roadmap-based methods

A roadmap is a graph embedded in $\mathcal{C}_{free}$. Planning:

1. construct graph vertices/configurations,
2. connect nearby feasible configurations,
3. connect start and goal,
4. run graph search.

Examples:

- visibility graph,
- Voronoi diagram,
- probabilistic roadmap (PRM),
- lattice/grid roadmap.

Grid and graph planning

Discretize environment into cells or graph nodes. Use:

- BFS for unweighted graph,
- Dijkstra for nonnegative costs,
- A* with heuristic,
- Theta*/any-angle variants for smoother paths,
- weighted A* for speed at cost of optimality.

Speedups:

- better heuristics,
- bidirectional search,
- hierarchical planning,
- jump point search on grids,
- contraction/hub labels in road networks,
- incremental replanning.

Dynamic environments: D* and D* Lite

D* algorithms replan efficiently when edge costs change.

D* Lite is an incremental heuristic search algorithm based on Lifelong Planning A*. It maintains:

- $g(s)$: current best path cost,
- $rhs(s)$: one-step lookahead cost.

A state is locally consistent if $g(s) = rhs(s)$. When costs change, update affected vertices and repair shortest-path tree instead of replanning from scratch.

Useful for robots discovering obstacles while moving.

Exam tip

D* and D* Lite are not just “A* again.” Their point is reuse: when the map changes, repair the previous shortest-path computation instead of starting from zero.

3. Informative Path Planning and Exploration

Robotic exploration

Goal: build a map or gather information in initially unknown environment while navigating safely.

Common decomposition:

1. perception and mapping,
2. frontier/information target generation,
3. utility estimation,
4. task allocation among robots,
5. path planning to chosen target,
6. execution and replanning.

Frontier-based exploration

A frontier is boundary between known free space and unknown space. Algorithm:

1. maintain occupancy grid,
2. detect frontier cells,
3. cluster frontiers,
4. score each frontier by utility:

$$U = \text{information gain} - \lambda \cdot \text{travel cost},$$

5. navigate to best frontier.

Variants:

- nearest frontier,
- largest expected information gain,
- cost-utility frontier,
- multi-robot frontier assignment,
- hierarchical frontier selection.

Example

If frontier F1 is 5 meters away and reveals about 20 unknown cells, while F2 is 20 meters away and reveals 30 cells, a utility score like **gain - cost** may choose F1 because it gives more information per travel effort.

Information-theoretic methods

Use uncertainty measures:

- entropy:

$$H(X) = - \sum_x p(x) \log p(x),$$

- mutual information:

$$I(X; Z) = H(X) - H(X|Z).$$

Choose paths maximizing expected information gain:

$$\pi^* = \arg \max_{\pi} \mathbb{E}[I(\text{map}; \text{observations along } \pi)] - \text{cost}(\pi).$$

This is often computationally expensive, so approximations and sampling are used.

Simple explanation

Frontier exploration asks “where can I go to see new space?” Information-theoretic exploration asks “where can I go to reduce uncertainty the most?” The second is more principled but usually more expensive.

Multi-robot task allocation

Centralized methods:

- one planner assigns all robots,
- can optimize globally,
- communication/computation bottleneck,
- single point of failure.

Decentralized methods:

- each robot decides using local communication,
- scalable and robust,
- may be suboptimal.

Algorithms:

- greedy assignment,
- Hungarian algorithm for linear assignment,
- auctions/market-based allocation,
- consensus-based bundle algorithm,
- distributed frontier selection.

4. Multi-Goal and Data-Collection Planning

Multi-goal planning

Robot must visit multiple goals/targets. This is related to TSP.

If travel cost from target i to j depends on order, heading, or local path constraints, it becomes sequence-dependent TSP.

Robotic complications:

- obstacles,
- start and final depot,
- motion constraints,
- sensing radius,
- profits/rewards,
- time/energy budget,
- multiple robots.

TSP variants in robotics

- TSP: visit all targets at minimum cost.
- ATSP: asymmetric costs.
- GTSP: choose one target from each cluster.
- TSPN: visit neighborhoods of targets, not exact points.
- Dubins TSP: curvature-constrained vehicle.
- Orienteering problem: maximize collected reward within budget.
- Team orienteering: multiple robots.
- Prize-collecting TSP: trade travel cost against missed-target penalties.

Curvature-constrained trajectories

Dubins vehicle constraints:

- moves forward,
- bounded curvature,
- state (x, y, θ) .

Shortest path between configurations consists of circular arcs and straight segments, with types such as LSL, RSR, LSR, RSL, RLR, LRL.

For data collection with sensing radius, the robot may only need to pass near a target. This yields Dubins TSP with neighborhoods (DTSPN).

Routing with profits

Data collection often has more candidate measurements than can be visited.

Formulation:

$$\max \sum_i p_i y_i$$

subject to:

$$travel_cost(route) \leq B, \quad y_i = 1 \text{ iff target } i \text{ is visited/observed.}$$

This models limited battery/time and target rewards.

Solution methods

Decoupled methods:

1. choose sequence of targets,
2. compute geometric/motion-feasible path between consecutive targets.

Transformation methods:

- transform neighborhoods or headings into sampled configurations,
- reduce to TSP/ATSP/GTSP,
- solve with combinatorial solvers.

Sampling-based methods:

- sample viewpoints/configurations,
- connect feasible transitions,
- search for high-reward path in sampled graph.

Tradeoff: decoupled methods are simple but may be suboptimal; integrated sampling captures constraints but can be expensive.

5. Sampling-Based Motion Planning

Motivation

Exact motion planning in high-dimensional continuous configuration spaces is hard. Sampling-based planners avoid explicit construction of $\mathcal{C}_{free}$ and use collision checking.

Probabilistic Roadmap (PRM)

Multi-query planner:

1. sample configurations in $\mathcal{C}_{free}$,
2. connect nearby samples with local planner if collision-free,
3. store roadmap,
4. answer queries by connecting start/goal and searching roadmap.

Best for static environments with many queries.

Rapidly-exploring Random Tree (RRT)

Single-query planner:

1. start tree at q_{start} ,
2. sample random q_{rand} ,
3. find nearest tree node q_{near} ,
4. extend from q_{near} toward q_{rand} ,
5. add new collision-free node,
6. stop when goal reached.

RRT explores large spaces quickly and handles kinodynamic constraints with appropriate steering.

Example

Starting at q_{start} , RRT samples a random configuration. It finds the nearest existing tree node and extends a short step toward the sample. Repeating this makes the tree rapidly spread into unexplored regions of configuration space.

RRT*

RRT* improves RRT with rewiring:

1. choose best parent among nearby nodes,
2. rewire neighbors through new node if cost improves.

It is asymptotically optimal under assumptions.

Probabilistic completeness

Planner is probabilistically complete if, when a feasible path exists,

$$\Pr[\text{planner finds a path by time/samples } n] \rightarrow 1$$

as $n \rightarrow \infty$.

PRM and RRT are probabilistically complete under standard assumptions.

Asymptotic optimality

Planner is asymptotically optimal if solution cost converges to optimal cost:

$$\Pr \left[\lim_{n \rightarrow \infty} c_n = c^* \right] = 1.$$

RRT is generally not asymptotically optimal. RRT* and PRM* are.

Simple explanation

Probabilistic completeness means a planner will eventually find a path if one exists. Asymptotic optimality is stronger: with enough samples, the path quality converges to the best possible path.

Practical considerations

Important design choices:

- sampling distribution,
- nearest-neighbor data structure,
- collision checker,
- local planner,
- goal bias,
- step size,
- smoothing/post-processing,
- handling differential constraints.

Exam Checklist

- Compare hierarchical, reactive, and hybrid robotic paradigms.
- Define embodiment, sensors, actuators, proprioceptive/exteroceptive sensing.
- Define configuration space and free/obstacle regions.
- Explain roadmap, grid, graph planning, A*, and speedups.
- Explain D* / D* Lite for changing environments.
- Explain frontier exploration, information gain, and entropy.
- Compare centralized and decentralized multi-robot task allocation.
- Define robotic TSP variants, TSPN, Dubins TSPN, and routing with profits.
- Compare decoupled, transformation, and sampling-based multi-goal methods.
- Explain PRM, RRT, RRT*, probabilistic completeness, and asymptotic optimality.